

Statistical and Machine Learning

Neural Networks for Tabular Data

Katarzyna Reluga

Neural Networks for Tabular Data

- ▶ Introduction
- ▶ Multilayer perceptron
- ▶ Backpropagation
- ▶ Training neural network
- ▶ Parallel training & regularization

Neural Networks for Tabular Data

- ▶ **Introduction**
- ▶ Multilayer perceptron
- ▶ Backpropagation
- ▶ Training neural network
- ▶ Parallel training & regularization

From Linear Models to More Flexible Models

- ▶ Many common models are **linear in the input $\mathbf{x}$** :
 - ▶ Binary logistic regression: $p(y \mid \mathbf{x}, \mathbf{w}) = \text{Ber}(y \mid \sigma(\mathbf{w}^T \mathbf{x}))$
 - ▶ Multiclass logistic regression:
 $p(y \mid \mathbf{x}, \mathbf{W}) = \text{Cat}(y \mid \text{softmax}(\mathbf{W}\mathbf{x}))$
 - ▶ Linear regression: $p(y \mid \mathbf{x}, \mathbf{w}) = \mathcal{N}(y \mid \mathbf{w}^T \mathbf{x}, \sigma^2)$
- ▶ A simple extension: transform the input before the linear step

$$f(\mathbf{x}) = \mathbf{W}\phi(\mathbf{x}) + \mathbf{b}$$

- ▶ For example, a polynomial transform in 1d:
 $\phi(\mathbf{x}) = [1, x, x^2, x^3, \dots]$
- ▶ This is called **basis function expansion**
- ▶ The model is still linear *in the parameters* $\theta = (\mathbf{W}, \mathbf{b}) \Rightarrow$ easy to fit
- ▶ Problem: ϕ must be designed by hand $\Rightarrow$ limiting

Deep Neural Networks: Learning Its Own Features and Stacking

- ▶ Idea: give the feature extractor its own parameters θ_2 to get $f(\mathbf{x}) = \mathbf{W} \phi(\mathbf{x}; \theta_2) + \mathbf{b}$, and $\theta = (\theta_1, \theta_2)$, $\theta_1 = (\mathbf{W}, \mathbf{b})$
 - ▶ now the features are learned from data, not fixed in advance
- ▶ Repeat this idea recursively, i.e., compose many simple functions, one feeding into the next

$$f(\mathbf{x}; \theta) = f_L(f_{L-1}(\cdots f_1(\mathbf{x}) \cdots))$$

- ▶ Each $f_\ell(\mathbf{x}) = f(\mathbf{x}; \theta_\ell)$ is a layer with its own parameters θ_ℓ
- ▶ This composition of layers is the key idea behind **deep neural networks (DNNs)**

Feedforward Neural Networks and Tabular Data

Feedforward Neural Networks

- ▶ More generally, a DNN composes differentiable functions into a **directed acyclic graph (DAG)** mapping input to output
- ▶ The simplest case: the DAG is just a chain $\Rightarrow$ a **feedforward neural network (FFNN)**, also called a **multilayer perceptron (MLP)**
- ▶ An MLP assumes the input is a fixed-size vector $\mathbf{x} \in \mathbb{R}^D$

Tabular Data

- ▶ Fixed-size vector inputs are typical of **tabular** (or **structured**) data
- ▶ Data stored as an $N \times D$ table, where each column has a clear meaning: height, weight, age, ...
- ▶ MLPs are a natural starting point for this kind of data

Looking Ahead

- ▶ Not all data comes in this neat tabular form
- ▶ **Unstructured data** (images, text): variable-sized input, individual elements often meaningless on their own
- ▶ Other architectures are designed for this:
 - ▶ **Convolutional neural networks (CNNs)** → images
 - ▶ **Recurrent neural networks (RNNs)** and **transformers** → sequences
 - ▶ **Graph neural networks (GNNs)** → graphs
- ▶ We will start with the simplest case: MLPs for tabular data

Neural Networks for Tabular Data

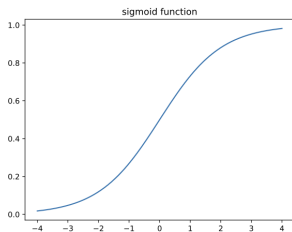
- ▶ Introduction
- ▶ **Multilayer perceptrons**
- ▶ Backpropagation
- ▶ Training neural network
- ▶ Parallel training & regularization

Perceptrons: From Single Units to Networks

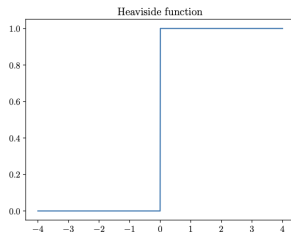
- ▶ A **perceptron** is a deterministic version of logistic regression
- ▶ It thresholds a linear combination of the inputs:

$$f(\mathbf{x}; \theta) = \mathbb{I}(\mathbf{w}^T \mathbf{x} + b \geq 0) = H(\mathbf{w}^T \mathbf{x} + b)$$

- ▶ H is the **Heaviside step function**: the output is exactly 0 or 1
- ▶ Single perceptrons are constrained by linear decision boundaries, which sharply limited early neural-network progress.



(a)



(b)

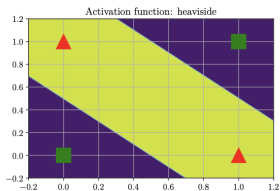
The XOR Problem

- ▶ **XOR** (exclusive OR) is the simplest example a single perceptron cannot solve: its two classes are not linearly separable

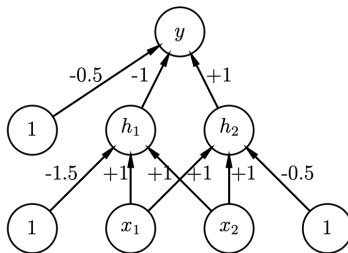
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

- ▶ Solution: stack perceptrons into a **multilayer perceptron (MLP)**

The XOR Problem: Solved with MLP



(a)



(b)

- ▶ Two **hidden units** build new features from the inputs: one computes AND (h_1), the other computes OR (h_2)
- ▶ The output combines these hidden features to reproduce XOR exactly
- ▶ Key insight: hidden units let the network construct intermediate features that the raw inputs don't have

Making MLPs Differentiable

- ▶ The Heaviside function is not differentiable $\Rightarrow$ this version of the MLP is hard to train, and is rarely used in practice
- ▶ Fix: replace it with a smooth **activation function** φ
- ▶ Each layer transforms the previous one:

$$\mathbf{z}_l = f_l(\mathbf{z}_{l-1}) = \varphi_l(\mathbf{b}_l + \mathbf{W}_l \mathbf{z}_{l-1}), \quad z_{kl} = \varphi_l \left(b_{kl} + \sum_{j=1}^{K_{l-1}} w_{lkj} z_{j,l-1} \right)$$

- ▶ The argument passed into φ is called the **pre-activation**:
 $\mathbf{a}_l = \mathbf{b}_l + \mathbf{W}_l \mathbf{z}_{l-1}$
- ▶ Differentiability lets us compute gradients layer by layer via the chain rule – this is **backpropagation**
- ▶ From now on, “MLP” refers to this differentiable version

Choosing an Activation Function

- ▶ Any differentiable φ works – except a *linear* one: stacked linear layers collapse into a single linear model, so nonlinearity is essential

$$\begin{aligned}f(\mathbf{x}; \boldsymbol{\theta}) &= \mathbf{W}_L c_L (\mathbf{W}_{L-1} c_{L-1} (\cdots (\mathbf{W}_1 \mathbf{x}) \cdots)) \\ &\propto \mathbf{W}_L \mathbf{W}_{L-1} \cdots \mathbf{W}_1 \mathbf{x} = \mathbf{W}' \mathbf{x}\end{aligned}$$

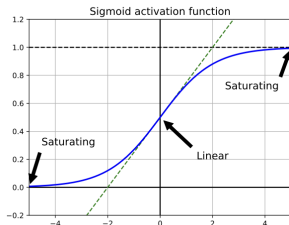
- ▶ Classic choice: **sigmoid** $\sigma(a) = \frac{1}{1+e^{-a}}$
 - ▶ smooth, but it **saturates** (becomes 0 or 1) for large $|a|$
 - ▶ $\Rightarrow$ gradients close to 0
 - ▶ $\Rightarrow$ any gradient signal from higher layers is not propagated back to earlier layers
 - ▶ $\Rightarrow$ deep networks become hard to train (the **vanishing gradient problem**)

Choosing an Activation Function

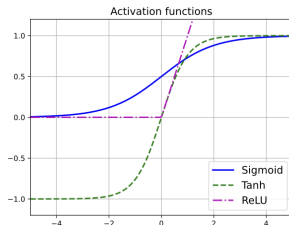
- Modern default: **rectified linear unit** or **ReLU**

$$\text{ReLU}(a) = \max(a, 0) = a\mathbb{I}(a \geq 0)$$

- ReLU “turns off” negative inputs, and passes positive inputs unchanged $\Rightarrow$ it does not saturate for positive inputs, which is one of the keys to training very deep networks



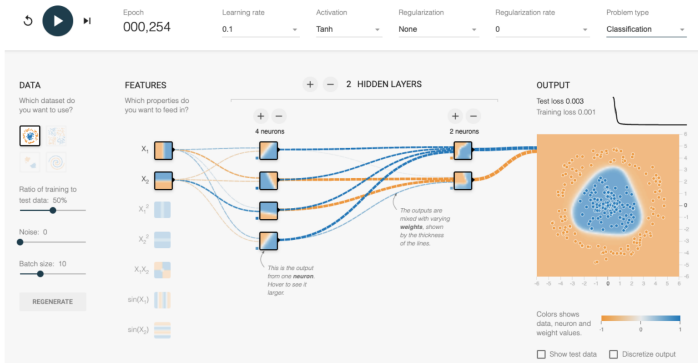
(a)



(b)

Example: Classifying Points in the Plane

- Task: separate two classes of 2d points with a highly nonlinear boundary (e.g., one class forming a ring around the other)



- You can explore this interactively at <http://playground.tensorflow.org>

Example: Classifying Points in the Plane

- ▶ A small MLP with two hidden layers handles this easily:

$$p(y \mid \mathbf{x}) = \text{Ber}(y \mid \sigma(\mathbf{w}_3^\top \mathbf{z}_2 + b_3))$$

$$\mathbf{z}_1 = \varphi(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1), \quad \mathbf{z}_2 = \varphi(\mathbf{W}_2 \mathbf{z}_1 + \mathbf{b}_2)$$

- ▶ Output layer a_3 combines 2 hidden units into a single logit score: $a_3 = \mathbf{w}_3^\top \mathbf{z}_2 + b_3$.
- ▶ Layer 2 takes a nonlinear combination of 4 hidden units and forms 2 new hidden units: $\mathbf{z}_2 = \varphi(\mathbf{W}_2 \mathbf{z}_1 + \mathbf{b}_2)$
 - ▶ Layer 1 takes the 2 input units and forms 4 hidden units: $\mathbf{z}_1 = \varphi(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1)$
- ▶ Each layer reshapes the input space a little further, until the classes become linearly separable
- ▶ Fitting all the parameters, $\theta = (\mathbf{W}_1, \mathbf{b}_1, \mathbf{W}_2, \mathbf{b}_2, \mathbf{w}_3, b_3)$, by minimizing the negative log-likelihood is enough to recover a highly nonlinear decision boundary

Example: Regression with Input-Dependent Noise

- ▶ Homoskedasticity in standard regression is often unrealistic
- ▶ **Heteroskedastic regression**: the predicted variance depends on the input $\Rightarrow$ model with two outputs, the predicted mean and the predicted standard deviation:

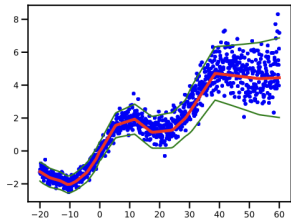
$$f_{\mu}(\mathbf{x}) = \mathbb{E}[y \mid \mathbf{x}, \boldsymbol{\theta}], \quad f_{\sigma}(\mathbf{x}) = \sqrt{\mathbb{V}[y \mid \mathbf{x}, \boldsymbol{\theta}]}$$

- ▶ Idea: a shared **backbone** extracts common features, feeding two separate **heads**:
 - ▶ μ head uses a **linear activation**, $\varphi(a) = a$, since the mean can take any value
 - ▶ σ head uses a **softplus activation**, $\varphi(a) = \sigma_+(a) = \log(1 + e^a)$, which keeps the output positive, as a standard deviation must be
- ▶ Putting it together:

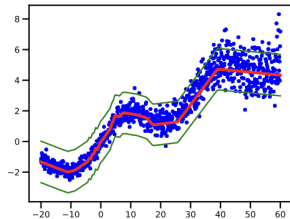
$$p(y \mid \mathbf{x}) = \mathcal{N}(y \mid \mathbf{w}_{\mu}^{\top} f(\mathbf{x}), \sigma_+(\mathbf{w}_{\sigma}^{\top} f(\mathbf{x})))$$

- ▶ Useful whenever the uncertainty itself changes across the input space, e.g. financial time series, or temperature trends under climate change

Illustration: Regression with Input-Dependent Noise



(a)



(b)

Predictions from an MLP fit using MLE to a 1d regression dataset with growing noise.

- a) Output variance is input-dependent.
- b) Mean is computed using same model as in (a), but output variance is treated as a fixed parameter σ^2 , which is estimated by MLE after training. Regression model is “unconfident” since it needs to adjust to the overall noise level, and cannot adapt to the noise at each point in the input.

Beyond Tabular Data: Images and Text

- ▶ **Images:** flatten the 2d pixel grid into a 1d vector, then feed it into an ordinary MLP (e.g., classifying handwritten digits)
- ▶ **Text:** convert a variable-length sequence of words into a fixed-size vector – embed each word, then average the embeddings – before feeding it into an MLP
- ▶ Both approaches work reasonably well, but throw away structure: pixel neighborhoods and word order are ignored
- ▶ In the next few lectures we will see architectures built specifically to exploit this structure

Why Depth Helps

- ▶ A single hidden layer is already a **universal approximator**: with enough hidden units, it can approximate any smooth function
- ▶ Yet in practice, deep networks consistently outperform shallow, wide ones
- ▶ Why: depth enables **hierarchical / compositional** features – later layers combine and reuse simpler patterns detected by earlier layers (cf. XOR example)
- ▶ Example: detecting a complex motif in a DNA sequence is easier if early layers first learn simple sub-motifs, and later layers combine them into the full pattern

The Deep Learning Revolution

- ▶ The core ideas behind DNNs are decades old, but they only became dominant in the 2010s
- ▶ First major success: automatic speech recognition
- ▶ The moment that captured the most attention: deep convolutional networks produced a dramatic, sudden jump in image classification accuracy
- ▶ Two key enablers:
 - ▶ Cheap **GPUs**, originally built for video games, massively speeding up the underlying computations
 - ▶ Much larger **labeled datasets**, allowing complex models to be fit without overfitting
- ▶ This fueled the rise of open-source deep learning libraries (TensorFlow, PyTorch, MXNet) that made these models broadly accessible

Neural Networks for Tabular Data

- ▶ Introduction
- ▶ Multilayer perceptrons
- ▶ **Backpropagation**
- ▶ Training neural network
- ▶ Parallel training & regularization

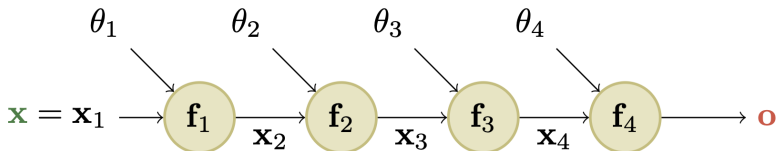
What Is Backpropagation?

- ▶ Training a network means minimizing a loss with respect to its parameters – which requires the gradient of the loss with respect to every parameter
- ▶ **Backpropagation** computes exactly this, by applying the chain rule of calculus layer by layer
- ▶ It was independently discovered several times starting in the 1960s, but only became widely used in the ML community in the mid-1980s
- ▶ In its most general form – applicable to any differentiable computation, not just a simple chain – it is called **automatic differentiation (autodiff)**

Composing Functions and the Chain Rule

- ▶ Consider a mapping $\mathbf{o} = \mathbf{f}(\mathbf{x})$, with $\mathbf{x} \in \mathbb{R}^n$ and $\mathbf{o} \in \mathbb{R}^m$, built by composing four functions: $\mathbf{f} = \mathbf{f}_4 \circ \mathbf{f}_3 \circ \mathbf{f}_2 \circ \mathbf{f}_1$
- ▶ Each piece maps between appropriately sized spaces:
 $\mathbf{f}_1 : \mathbb{R}^n \rightarrow \mathbb{R}^{m_1}$, $\mathbf{f}_2 : \mathbb{R}^{m_1} \rightarrow \mathbb{R}^{m_2}$, $\mathbf{f}_3 : \mathbb{R}^{m_2} \rightarrow \mathbb{R}^{m_3}$,
 $\mathbf{f}_4 : \mathbb{R}^{m_3} \rightarrow \mathbb{R}^m$
- ▶ The intermediate values needed to compute $\mathbf{o}$ are: $\mathbf{x}_2 = \mathbf{f}_1(\mathbf{x})$, $\mathbf{x}_3 = \mathbf{f}_2(\mathbf{x}_2)$, $\mathbf{x}_4 = \mathbf{f}_3(\mathbf{x}_3)$, $\mathbf{o} = \mathbf{f}_4(\mathbf{x}_4)$
- ▶ By the chain rule, the Jacobian of the whole mapping is a product of the Jacobians of each piece:

$$\frac{\partial \mathbf{o}}{\partial \mathbf{x}} = \frac{\partial \mathbf{o}}{\partial \mathbf{x}_4} \frac{\partial \mathbf{x}_4}{\partial \mathbf{x}_3} \frac{\partial \mathbf{x}_3}{\partial \mathbf{x}_2} \frac{\partial \mathbf{x}_2}{\partial \mathbf{x}} = \mathbf{J}_{f_4}(\mathbf{x}_4) \mathbf{J}_{f_3}(\mathbf{x}_3) \mathbf{J}_{f_2}(\mathbf{x}_2) \mathbf{J}_{f_1}(\mathbf{x})$$



The Jacobian Matrix

- ▶ The Jacobian collects partial derivatives of $\mathbf{f}$ into a matrix:

$$\mathbf{J}_f(\mathbf{x}) = \frac{\partial \mathbf{f}(\mathbf{x})}{\partial \mathbf{x}} = \begin{pmatrix} \nabla f_1(\mathbf{x})^\top \\ \vdots \\ \nabla f_m(\mathbf{x})^\top \end{pmatrix} = \left(\frac{\partial \mathbf{f}}{\partial x_1}, \dots, \frac{\partial \mathbf{f}}{\partial x_n} \right) \in \mathbb{R}^{m \times n}$$

- ▶ i -th row of $\mathbf{J}_f(\mathbf{x})$ is the gradient of the i -th output, $\nabla f_i(\mathbf{x})^\top$; it can be extracted with a **vector-Jacobian product (VJP)**: $\mathbf{e}_i^\top \mathbf{J}_f(\mathbf{x})$, where $\mathbf{e}_i \in \mathbb{R}^m$ is a standard basis vector
- ▶ j -th column of $\mathbf{J}_f(\mathbf{x})$ collects the derivatives of every output with respect to x_j , it can be extracted with a **Jacobian-vector product (JVP)**: $\mathbf{J}_f(\mathbf{x}) \mathbf{e}_j$, where $\mathbf{e}_j \in \mathbb{R}^n$
- ▶ Computing the full Jacobian reduces to either n JVPs or m VJPs – and we want to pick whichever is cheaper
- ▶ Special case $m = 1$ (scalar output): the gradient $\nabla f(\mathbf{x})$ is a column vector with the same shape as $\mathbf{x}$, while the Jacobian is a row vector, so $\nabla f(\mathbf{x}) = \mathbf{J}_f(\mathbf{x})^\top$

Forward Mode

- ▶ If $n < m$: build the product $\mathbf{J}_f(\mathbf{x})\mathbf{v}$ starting from the innermost function and working outward:

$$\underbrace{\mathbf{J}_{f_4}(\mathbf{x}_4)}_{m \times m_3} \underbrace{\mathbf{J}_{f_3}(\mathbf{x}_3)}_{m_3 \times m_2} \underbrace{\mathbf{J}_{f_2}(\mathbf{x}_2)}_{m_2 \times m_1} \underbrace{\mathbf{J}_{f_1}(\mathbf{x}_1)}_{m_1 \times n} \underbrace{\mathbf{v}}_{n \times 1}$$

- ▶ Every step is a matrix-vector product – **forward mode** differentiation follows the layers in their natural order, $k = 1, \dots, K$

Algorithm: Forward Mode Differentiation

Input: $\mathbf{x}$

1. $\mathbf{x}_1 := \mathbf{x}$
2. $\mathbf{v}_j := \mathbf{e}_j \in \mathbb{R}^n$ for $j = 1, \dots, n$
3. **for** $k = 1, \dots, K$ **do**
 - ▶ $\mathbf{x}_{k+1} = \mathbf{f}_k(\mathbf{x}_k)$
 - ▶ $\mathbf{v}_j := \mathbf{J}_{f_k}(\mathbf{x}_k) \mathbf{v}_j$ for $j = 1, \dots, n$
4. **Return** $\mathbf{o} = \mathbf{x}_{K+1}$, $[\mathbf{J}_f(\mathbf{x})]_{:,j} = \mathbf{v}_j$ for $j = 1, \dots, n$

Reverse Mode

- ▶ If $n > m$ (e.g. a scalar loss): build the product $\mathbf{u}^\top \mathbf{J}_f(\mathbf{x})$ starting from the outermost function and working inward – **reverse mode** differentiation follows the layers in reverse, $k = K, \dots, 1$
- ▶ Neural network training almost always has a scalar loss as its output $\Rightarrow$ reverse mode is the natural choice – this is exactly backpropagation

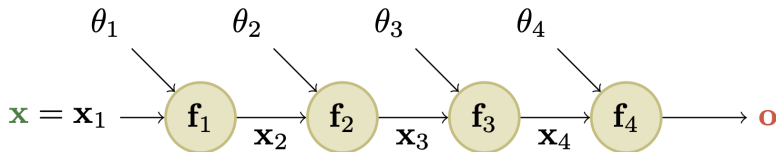
Algorithm: Reverse Mode Differentiation

Input: $\mathbf{x}$

1. $\mathbf{x}_1 := \mathbf{x}$
2. **for** $k = 1, \dots, K$ **do:** $\mathbf{x}_{k+1} = \mathbf{f}_k(\mathbf{x}_k)$
3. $\mathbf{u}_i := \mathbf{e}_i \in \mathbb{R}^m$ for $i = 1, \dots, m$
4. **for** $k = K, \dots, 1$ **do:** $\mathbf{u}_i^\top := \mathbf{u}_i^\top \mathbf{J}_{f_k}(\mathbf{x}_k)$
5. **Return** $\mathbf{o} = \mathbf{x}_{K+1}$, $[\mathbf{J}_f(\mathbf{x})]_{i,:} = \mathbf{u}_i^\top$ for $i = 1, \dots, m$

Parameterized Layers

- ▶ So far, the layers in our chain had no learnable parameters
- ▶ Real networks attach (optional) parameters $\theta_1, \dots, \theta_K$ to each layer
- ▶ Notation: $\mathbf{f}_k(\mathbf{x}_k, \theta_k)$ is the function at layer k , where $\mathbf{x}_k$ is the output of the previous layer and θ_k are that layer's (possibly empty) parameters
- ▶ We focus on the case where the whole mapping is a scalar loss, $\mathcal{L} : \mathbb{R}^n \rightarrow \mathbb{R}$, since this is what we minimize during training



Example: A One-Hidden-Layer MLP as a Chain

- ▶ Consider the squared-error loss for an MLP with one hidden layer:

$$\mathcal{L}((\mathbf{x}, \mathbf{y}), \boldsymbol{\theta}) = \frac{1}{2} \|\mathbf{y} - \mathbf{W}_2 \varphi(\mathbf{W}_1 \mathbf{x})\|_2^2$$

- ▶ This is exactly a chain of four simple layers:

$$\mathbf{x}_2 = \mathbf{f}_1(\mathbf{x}, \boldsymbol{\theta}_1) = \mathbf{W}_1 \mathbf{x}, \quad \mathbf{x}_3 = \mathbf{f}_2(\mathbf{x}_2, \emptyset) = \varphi(\mathbf{x}_2)$$

$$\mathbf{x}_4 = \mathbf{f}_3(\mathbf{x}_3, \boldsymbol{\theta}_3) = \mathbf{W}_2 \mathbf{x}_3, \quad \mathcal{L} = \mathbf{f}_4(\mathbf{x}_4, \mathbf{y}) = \frac{1}{2} \|\mathbf{x}_4 - \mathbf{y}\|^2$$

- ▶ Layer 2 (the nonlinearity) has no parameters of its own:
 $\boldsymbol{\theta}_2 = \emptyset$
- ▶ Since the final output is again a scalar loss, reverse mode is the efficient choice

Propagating Gradients Backward Through the Layers

- ▶ We already know the gradient of the loss with respect to the final layer's output, $\partial\mathcal{L}/\partial\mathbf{x}_4$.
- ▶ The chain rule gives the gradients with respect to the parameters at every layer:

$$\frac{\partial\mathcal{L}}{\partial\theta_3} = \frac{\partial\mathcal{L}}{\partial\mathbf{x}_4} \frac{\partial\mathbf{x}_4}{\partial\theta_3}, \quad \frac{\partial\mathcal{L}}{\partial\theta_2} = \frac{\partial\mathcal{L}}{\partial\mathbf{x}_4} \frac{\partial\mathbf{x}_4}{\partial\mathbf{x}_3} \frac{\partial\mathbf{x}_3}{\partial\theta_2}, \quad \frac{\partial\mathcal{L}}{\partial\theta_1} = \frac{\partial\mathcal{L}}{\partial\mathbf{x}_4} \frac{\partial\mathbf{x}_4}{\partial\mathbf{x}_3} \frac{\partial\mathbf{x}_3}{\partial\mathbf{x}_2} \frac{\partial\mathbf{x}_2}{\partial\theta_1}.$$

- ▶ Each $\partial\mathcal{L}/\partial\theta_k$ is a d_k -dimensional gradient row vector, where d_k is the number of parameters in layer k .
- ▶ These gradients can be computed **recursively**. With the row-vector convention, $\frac{\partial\mathcal{L}}{\partial\mathbf{x}_{k-1}} = \frac{\partial\mathcal{L}}{\partial\mathbf{x}_k} \frac{\partial\mathbf{x}_k}{\partial\mathbf{x}_{k-1}}$, where $\partial\mathbf{x}_k/\partial\mathbf{x}_{k-1}$ is an $n_k \times n_{k-1}$ Jacobian matrix.
- ▶ The parameter gradient at layer k is then computed as $\frac{\partial\mathcal{L}}{\partial\theta_k} = \frac{\partial\mathcal{L}}{\partial\mathbf{x}_{k+1}} \frac{\partial\mathbf{x}_{k+1}}{\partial\theta_k}$. This recursion – propagating gradients backward one layer at a time – is **backpropagation**.

Algorithm: Backpropagation for an MLP

Input: $\mathbf{x}$, parameters $\theta_1, \dots, \theta_K$

Forward pass:

1. $\mathbf{x}_1 := \mathbf{x}$
2. **for** $k = 1, \dots, K$ **do:** $\mathbf{x}_{k+1} = \mathbf{f}_k(\mathbf{x}_k, \theta_k)$

Backward pass:

3. $\mathbf{u}_{K+1} := 1$
4. **for** $k = K, \dots, 1$ **do:**
 - ▶ $\mathbf{g}_k := \mathbf{u}_{k+1}^\top \frac{\partial \mathbf{f}_k(\mathbf{x}_k, \theta_k)}{\partial \theta_k}$
 - ▶ $\mathbf{u}_k^\top := \mathbf{u}_{k+1}^\top \frac{\partial \mathbf{f}_k(\mathbf{x}_k, \theta_k)}{\partial \mathbf{x}_k}$

Return: $\mathcal{L} = \mathbf{x}_{K+1}$, $\nabla_{\mathbf{x}} \mathcal{L} = \mathbf{u}_1$, $\{\nabla_{\theta_k} \mathcal{L} = \mathbf{g}_k : k = 1, \dots, K\}$

What Backprop Also Gives You for Free

- ▶ Besides the gradient with respect to every parameter, the same recursion also yields the gradient of the loss with respect to the **input**, $\nabla_x \mathcal{L}$
- ▶ This isn't needed to update parameters, but it is useful elsewhere – e.g. for constructing or modifying inputs to change the model's output in a desired way
- ▶ All that remains is to specify how to compute the VJP for each kind of layer

VJP for the Cross-Entropy Layer

- ▶ This layer takes logits $\mathbf{x}$ and a target label distribution $\mathbf{y}$, and returns a scalar loss:

$$z = f(\mathbf{x}) = - \sum_c y_c \log(\text{softmax}(\mathbf{x})_c) = - \sum_c y_c \log p_c,$$

where $\mathbf{p} = \text{softmax}(\mathbf{x}) = \frac{e^{x_{c'}}}{\sum_{c'=1}^C e^{x_{c'}}}$

- ▶ Despite the nonlinear softmax inside, the Jacobian with respect to the logits has a remarkably simple closed form:

$$\mathbf{J} = \frac{\partial z}{\partial \mathbf{x}} = (\mathbf{p} - \mathbf{y})^\top \in \mathbb{R}^{1 \times C}$$

- ▶ In words: predicted probabilities minus the true label – one reason cross-entropy is the default loss for classification

VJP for an Elementwise Nonlinearity

- ▶ For a layer applying an activation elementwise, $\mathbf{z} = \varphi(\mathbf{x})$, i.e. $z_i = \varphi(x_i)$, the Jacobian is diagonal:

$$\mathbf{J} = \frac{\partial \mathbf{z}}{\partial \mathbf{x}} = \text{diag}(\varphi'(\mathbf{x}))$$

- ▶ So the VJP reduces to a simple elementwise product, $\mathbf{u}^\top \mathbf{J} = \mathbf{u} \odot \varphi'(\mathbf{x})$ – no matrix multiply needed
- ▶ For ReLU, $\varphi(a) = \max(a, 0)$, the derivative is a 0/1 mask:

$$\varphi'(a) = \begin{cases} 1 & a > 0 \\ 0 & a < 0 \end{cases},$$

- ▶ Hence $\text{ReLU}'(a) = H(a)$ a Heavyside step function, i.e., the upstream gradient passes through wherever the input was positive, and is blocked otherwise.

VJP for a Linear Layer: the Input

- ▶ For $\mathbf{z} = \mathbf{f}(\mathbf{x}, \mathbf{W}) = \mathbf{W}\mathbf{x}$, with $\mathbf{W} \in \mathbb{R}^{m \times n}$, write each output coordinate as $z_i = \sum_{k=1}^n W_{ik} x_k$
- ▶ Differentiating with respect to (wrt) a single input x_j , only the $k = j$ term survives:

$$\frac{\partial z_i}{\partial x_j} = \sum_{k=1}^n W_{ik} \frac{\partial x_k}{\partial x_j} = W_{ij}$$

- ▶ So the Jacobian wrt the input is simply the weight matrix itself:

$$\mathbf{J} = \frac{\partial \mathbf{z}}{\partial \mathbf{x}} = \mathbf{W}$$

- ▶ Hence the VJP wrt the input is just a vector-matrix product:

$$\mathbf{u}^\top \mathbf{J} = \mathbf{u}^\top \mathbf{W} \in \mathbb{R}^{1 \times n}$$

VJP for a Linear Layer: the Weights

- Differentiate wrt a single weight W_{ij} . Writing $z_k = \sum_{l=1}^n W_{kl}x_l$:

$$\frac{\partial z_k}{\partial W_{ij}} = \sum_{l=1}^n x_l \mathbb{I}(i = k \text{ and } j = l) = x_j \mathbb{I}(i = k)$$

- So $\partial \mathbf{z} / \partial W_{ij}$ is a vector that is zero everywhere except for an x_j sitting in position i
- Taking the VJP with this vector, only the $k = i$ term in the sum over $\mathbf{u}$ survives:

$$\mathbf{u}^\top \frac{\partial \mathbf{z}}{\partial W_{ij}} = \sum_{k=1}^m u_k \frac{\partial z_k}{\partial W_{ij}} = u_i x_j$$

- Stacking this over all (i, j) gives the gradient for the *whole* weight matrix as a clean outer product:

$$\mathbf{u}^\top \frac{\partial \mathbf{z}}{\partial \mathbf{W}} = \mathbf{u} \mathbf{x}^\top \in \mathbb{R}^{m \times n}$$

- These two simple rules are all that is needed to backpropagate through the linear layers that make up most of an MLP

From Chains to Computation Graphs

- ▶ MLPs are the simplest case: each layer feeds directly into the next, forming a chain
- ▶ General DNNs combine differentiable building blocks in much richer ways, forming a **computation graph**
- ▶ The only requirement: the graph must be a **directed acyclic graph (DAG)** – no cycles allowed
- ▶ This idea is sometimes called **differentiable programming**: building complex functions out of simple, differentiable pieces, the same way programmers compose ordinary functions

A Worked Example

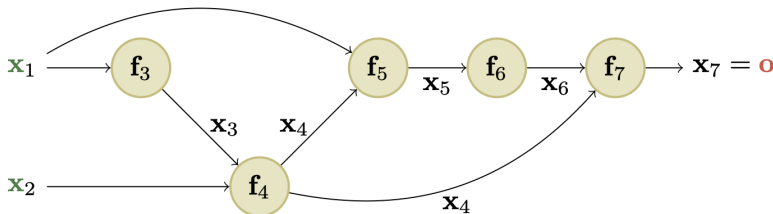
- Consider the function:

$$f(x_1, x_2) = x_2 e^{x_1} \sqrt{x_1 + x_2 e^{x_1}}$$

- We can compute it via a sequence of simple, differentiable steps, numbered in topological order (parents before children):

$$x_3 = e^{x_1}, \quad x_4 = x_2 x_3, \quad x_5 = x_1 + x_4, \quad x_6 = \sqrt{x_5}, \quad x_7 = x_4 x_6$$

- The output is $o = x_7$
- Notice that x_4 feeds into *two* later computations: directly into x_7 , and indirectly through $x_5 \rightarrow x_6 \rightarrow x_7$



When Paths Split and Merge

- ▶ In a DAG, a single intermediate value can feed into more than one later computation – exactly what happens with x_4 above
- ▶ If a node influences the final output through multiple paths, its gradient must **sum the contributions from every path** it feeds into:

$$\frac{\partial o}{\partial x_4} = \frac{\partial o}{\partial x_5} \frac{\partial x_5}{\partial x_4} + \frac{\partial o}{\partial x_7} \frac{\partial x_7}{\partial x_4}$$

- ▶ Working in reverse topological order (from output back to input) ensures each node's gradient is computed only once, even when many paths merge back together

Backward Through the Example, Step by Step

- ▶ Starting from the output and working backward, each step only needs the adjoint already computed at its children:

$$\frac{\partial o}{\partial x_7} = \mathbf{1}_m, \quad \frac{\partial o}{\partial x_6} = \frac{\partial o}{\partial x_7} \frac{\partial x_7}{\partial x_6}, \quad \frac{\partial o}{\partial x_5} = \frac{\partial o}{\partial x_6} \frac{\partial x_6}{\partial x_5}$$

- ▶ And finally, at the branching node, we sum both incoming contributions:

$$\frac{\partial o}{\partial x_4} = \frac{\partial o}{\partial x_5} \frac{\partial x_5}{\partial x_4} + \frac{\partial o}{\partial x_7} \frac{\partial x_7}{\partial x_4}$$

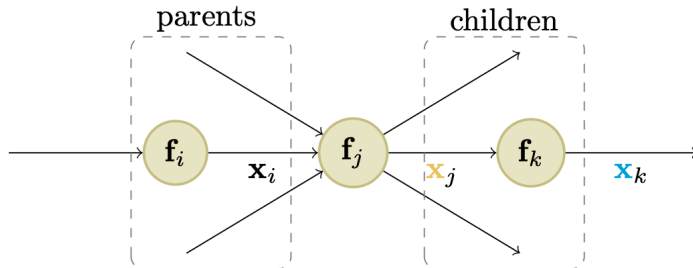
- ▶ By the time we reach x_4 , the gradients at both of its children, x_5 and x_7 , are already available – we simply look them up and combine them

The General Backpropagation Rule

- ▶ For any node j in the graph, its gradient is a sum over all of its children k :

$$\frac{\partial \mathbf{o}}{\partial \mathbf{x}_j} = \sum_{k \in \text{children}(j)} \frac{\partial \mathbf{o}}{\partial \mathbf{x}_k} \frac{\partial \mathbf{x}_k}{\partial \mathbf{x}_j}$$

- ▶ The term $\partial \mathbf{o} / \partial \mathbf{x}_k$ – already computed for every child – is called the **adjoint**
- ▶ This single rule – the one we just applied by hand at x_4 – is what makes backpropagation work for any DAG



Static vs Dynamic Graphs

- ▶ The computation graph can be built **ahead of time**, fixed before any data is seen – a **static graph**
- ▶ Or it can be built **just in time**, by tracing the actual computation as it runs on a given input – enabling a **dynamic graph**, whose shape can depend on the input itself
- ▶ Most modern deep learning libraries default to the dynamic, trace-based approach, since it is more flexible and easier to debug

Neural Networks for Tabular Data

- ▶ Introduction
- ▶ Multilayer perceptrons
- ▶ Backpropagation
- ▶ **Training neural networks**
- ▶ Parallel training & regularization

Training Neural Networks: The Basic Recipe

- ▶ As with other statistical models, we fit a DNN by **maximum likelihood**, i.e. by minimizing the negative log-likelihood:

$$\mathcal{L}(\theta) = -\log p(\mathcal{D} \mid \theta) = -\sum_{n=1}^N \log p(\mathbf{y}_n \mid \mathbf{x}_n; \theta)$$

- ▶ A regularizer (e.g. a negative log prior) is often added on top – more on this later
- ▶ In principle this is simple: use backpropagation to get the gradient, then hand it to an off-the-shelf optimizer (**Adam** is a popular default: it scales to large datasets and converges quickly using momentum and per-parameter step sizes)
- ▶ In practice it's not so simple – the rest of this section covers what can go wrong, and how to fix it
- ▶ The DNN loss is **non-convex**, so we are not guaranteed to find the global optimum – yet gradient descent often finds surprisingly good solutions anyway

Tuning the Learning Rate

- ▶ The learning rate (step size) used by the optimizer has to be tuned carefully
- ▶ Too large, and training can diverge or oscillate; too small, and training crawls or gets stuck
- ▶ Getting this right is often the single most important practical choice when training a DNN

The Vanishing and Exploding Gradient Problem

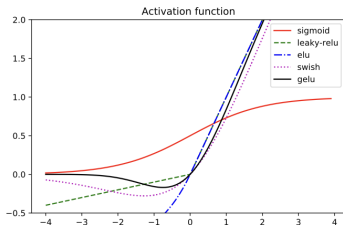
- ▶ In very deep models, the gradient signal passed backward through many layers tends to become either very small or very large as it propagates
- ▶ Consider the gradient of the loss with respect to the hidden units at layer l :

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}_l} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}_{l+1}} \frac{\partial \mathbf{z}_{l+1}}{\partial \mathbf{z}_l} = \mathbf{J}_l \mathbf{g}_{l+1}$$

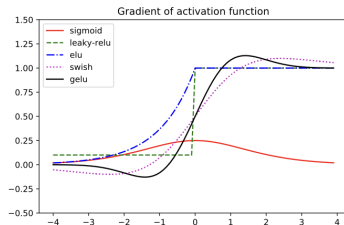
- ▶ If the Jacobian $\mathbf{J}_l$ were the same at every layer, the contribution of the gradient at the final layer, $\mathbf{g}_L$, to layer l would be $\mathbf{J}^{L-l} \mathbf{g}_L$
- ▶ So the behavior depends entirely on the **spectral radius** λ of $\mathbf{J}$ (its largest eigenvalue magnitude): $\lambda > 1 \Rightarrow$ gradients can **explode**; $\lambda < 1 \Rightarrow$ gradients can **vanish**

Illustration: Activation Functions and their Gradients

Name	Definition	Range
Sigmoid	$\sigma(a) = \frac{1}{1+e^{-a}}$	$[0, 1]$
Hyperbolic tangent	$\tanh(a) = 2\sigma(2a) - 1$	$[-1, 1]$
Softplus	$\sigma_+(a) = \log(1 + e^a)$	$[0, \infty]$
Rectified linear unit	$\text{ReLU}(a) = \max(a, 0)$	$[0, \infty]$
Leaky ReLU	$\max(a, 0) + \alpha \min(a, 0)$	$[-\infty, \infty]$
Exponential linear unit	$\max(a, 0) + \min(\alpha(e^a - 1), 0)$	$[-\infty, \infty]$
Swish	$a\sigma(a)$	$[-\infty, \infty]$
GELU	$a\Phi(a)$	$[-\infty, \infty]$



(a)



(b)

Gradient Clipping

- ▶ Exploding gradients have a simple fix: **gradient clipping**, which caps the magnitude of the gradient without changing its direction:

$$\mathbf{g}' = \min\left(1, \frac{c}{\|\mathbf{g}\|}\right) \mathbf{g}$$

- ▶ The norm of $\mathbf{g}'$ can never exceed c
- ▶ Vanishing gradients are harder to fix – there is no single trick, but several complementary strategies:
 - ▶ Use **non-saturating activation functions**
 - ▶ Make updates **additive** rather than multiplicative (residual connections)
 - ▶ **Standardize** the activations at each layer so their distribution stays stable during training
 - ▶ Choose **initial parameter values** carefully

Why Sigmoid Saturates

- ▶ Consider a layer computing $\mathbf{z} = \sigma(\mathbf{W}\mathbf{x})$, with the sigmoid activation:

$$\varphi(a) = \sigma(a) = \frac{1}{1 + \exp(-a)}$$

- ▶ If the weights are initialized large (positive or negative), $a = \mathbf{W}\mathbf{x}$ easily takes on large values, pushing $\mathbf{z}$ to saturate near 0 or 1
- ▶ The derivative of the sigmoid is:

$$\varphi'(a) = \sigma(a)(1 - \sigma(a))$$

- ▶ This is close to zero exactly when $\sigma(a)$ is close to 0 or 1 – i.e. exactly in the saturated regime

Vanishing Gradients Through a Sigmoid Layer

- ▶ Combining the sigmoid's Jacobian with the linear layer's Jacobian gives the gradient with respect to the input:

$$\frac{\partial \mathbf{z}}{\partial \mathbf{x}} = \text{diag}(\mathbf{z}(1 - \mathbf{z})) \mathbf{W}$$

- ▶ And with respect to the weights:

$$\frac{\partial \mathbf{z}}{\partial \mathbf{W}} = \mathbf{z}(1 - \mathbf{z}) \mathbf{x}^\top$$

- ▶ In both cases, if $\mathbf{z}$ is near 0 or 1, the gradient goes to zero – this is exactly the vanishing gradient problem
- ▶ This motivates **non-saturating activation functions**, which keep a useful gradient signal even far from the origin

ReLU: A Non-Saturating Alternative

- ▶ The most common non-saturating activation is the **rectified linear unit (ReLU)**:

$$\text{ReLU}(a) = \max(a, 0) = a \mathbb{I}(a > 0), \quad \text{ReLU}'(a) = \mathbb{I}(a > 0)$$

- ▶ For a layer computing $\mathbf{z} = \text{ReLU}(\mathbf{W}\mathbf{x})$, the Jacobians become:

$$\frac{\partial \mathbf{z}}{\partial \mathbf{x}} = \mathbf{W}^\top \mathbb{I}(\mathbf{z} > \mathbf{0}), \quad \frac{\partial \mathbf{z}}{\partial \mathbf{W}} = \mathbb{I}(\mathbf{z} > \mathbf{0}) \mathbf{x}^\top$$

- ▶ As long as $\mathbf{z}$ stays positive, the gradient does **not** vanish

The Dead ReLU Problem

- ▶ ReLU isn't free of problems: if the weights are initialized large and negative, $\mathbf{a} = \mathbf{W}\mathbf{x}$ easily becomes very negative, pushing $\mathbf{z}$ to exactly 0
- ▶ Once $\mathbf{z} = 0$, the gradient with respect to the weights is also exactly 0 – so those units can never recover
- ▶ This is known as the **dead ReLU** problem: some hidden units stay permanently switched off

Non-Saturating Variants of ReLU

- ▶ **Leaky ReLU**: keeps a small slope $\alpha \in (0, 1)$ for negative inputs, so a little signal always gets through:

$$\text{LReLU}(a; \alpha) = \max(\alpha a, a)$$

- ▶ If α is learned rather than fixed, this is called **parametric ReLU**
- ▶ **ELU**, a smooth alternative:

$$\text{ELU}(a; \alpha) = \begin{cases} \alpha(e^a - 1) & a \leq 0 \\ a & a > 0 \end{cases}$$

- ▶ **SELU** ($= \lambda \text{ELU}(a; \alpha)$): with carefully chosen α, λ , this provably keeps each layer's output standardized, even without a separate normalization step

Residual Connections

- ▶ Another fix for vanishing gradients: a **residual network (ResNet)**, where each layer is a **residual block**:

$$\mathcal{F}'_l(\mathbf{x}) = \mathcal{F}_l(\mathbf{x}) + \mathbf{x}$$

- ▶ $\mathcal{F}_l$ only has to learn the *residual* – the small correction needed to turn the input into the desired output – which is often easier than learning the full mapping from scratch
- ▶ Stacking many such blocks, the activations at the output layer can be written in terms of any earlier layer l :

$$\mathbf{z}_L = \mathbf{z}_l + \sum_{i=l}^{L-1} \mathcal{F}_i(\mathbf{z}_i; \boldsymbol{\theta}_i)$$

- ▶ This gives gradients a direct, unobstructed path from the output all the way back to any earlier layer

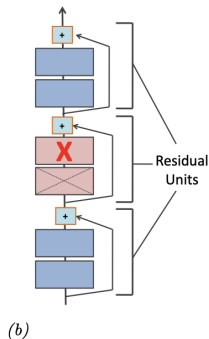
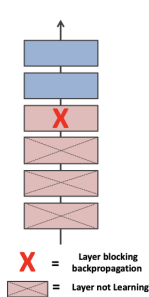
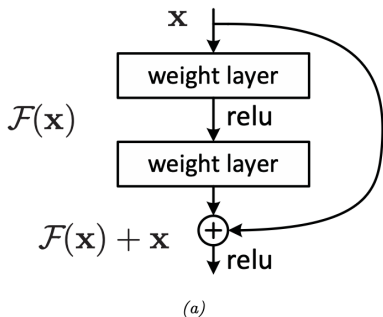
Why Residual Connections Fix Vanishing Gradients

- ▶ Differentiating the previous expression with respect to the parameters of layer l , and applying the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \theta_l} = \frac{\partial \mathbf{z}_l}{\partial \theta_l} \frac{\partial \mathcal{L}}{\partial \mathbf{z}_L} \left(1 + \sum_{i=l}^{L-1} \frac{\partial \mathcal{F}_i(\mathbf{z}_i; \theta_i)}{\partial \mathbf{z}_l} \right) = \frac{\partial \mathbf{z}_l}{\partial \theta_l} \frac{\partial \mathcal{L}}{\partial \mathbf{z}_L} + \text{other terms}$$

- ▶ The key term, $\partial \mathbf{z}_l / \partial \theta_l \cdot \partial \mathcal{L} / \partial \mathbf{z}_L$, connects layer l directly to the gradient at the final layer L – with no multiplicative chain of Jacobians in between
- ▶ This direct path is **independent of depth**, which is why residual networks can be trained successfully even with hundreds of layers

Illustration: Residual Connections



- a) Illustration of a residual block.
- (b) Illustration of why adding residual connections can help when training a very deep model.

Parameter Initialization Matters

- ▶ Since the training objective is non-convex, the **initial values** of the parameters strongly affect which solution we end up at, and how easily gradients flow forward and backward through the model
- ▶ The rest of this section covers some common heuristics for choosing a good starting point

Keeping Activations Stable: A Variance Argument

- ▶ Consider a single linear unit, $o_i = \sum_{j=1}^{n_{\text{in}}} w_{ij} x_j$, with $w_{ij} \sim \mathcal{N}(0, \sigma^2)$ and inputs with $\mathbb{E}[x_j] = 0$, $\mathbb{V}[x_j] = \gamma^2$, independent of the weights
- ▶ The output then has:

$$\mathbb{E}[o_i] = 0, \quad \mathbb{V}[o_i] = n_{\text{in}} \sigma^2 \gamma^2$$

- ▶ To stop the variance from blowing up as it passes through many layers, we need $n_{\text{in}} \sigma^2 = 1$, where n_{in} is the **fan-in** (number of incoming connections)
- ▶ By the same argument applied to the backward pass, we'd also like $n_{\text{out}} \sigma^2 = 1$, where n_{out} is the **fan-out**

Balancing Forward and Backward Passes: Xavier/Glorot Initialization

- ▶ Satisfying both the fan-in and fan-out conditions at once leads to:

$$\sigma^2 = \frac{2}{n_{\text{in}} + n_{\text{out}}}$$

- ▶ This is known as **Xavier** or **Glorot initialization**
- ▶ Two special cases: $\sigma^2 = 1/n_{\text{in}}$ is **LeCun initialization** (equivalent to Glorot when $n_{\text{in}} = n_{\text{out}}$); $\sigma^2 = 2/n_{\text{in}}$ is **He initialization**
- ▶ The same idea works for a uniform distribution, $w_{ij} \sim \text{Unif}(-a, a)$, by setting $a = \sqrt{6/(n_{\text{in}} + n_{\text{out}})}$
- ▶ Rule of thumb: Glorot for linear/tanh/logistic/softmax units, He for ReLU and its variants, LeCun for SELU

Data-Driven Initialization: LSUV

- ▶ Rather than relying purely on theory, **layer-sequential unit-variance (LSUV)** initialization adapts to the actual data:
 - ▶ Initialize each layer's weights as an **orthonormal matrix** (draw from a standard Gaussian, then orthonormalize via QR or SVD)
 - ▶ Pass one minibatch through the network, and for each layer l , measure the variance v_l of its activations
 - ▶ Rescale that layer's weights: $\mathbf{W}_l := \mathbf{W}_l / \sqrt{v_l}$
- ▶ This behaves like an orthonormal initialization combined with a one-off normalization step computed from real data – faster than full per-batch normalization, but can work just as well in practice

Neural Networks for Tabular Data

- ▶ Introduction
- ▶ Multilayer perceptrons
- ▶ Backpropagation
- ▶ Training neural networks
- ▶ **Parallel training & regularization**

Parallel Training

- ▶ Training large models on large datasets can be slow – specialized hardware (GPUs, TPUs) helps, and so does spreading the work across multiple machines
- ▶ **Model parallelism**: split the model itself across machines (powerful, but requires tight communication)
- ▶ **Data parallelism**: every machine keeps a full copy of the model, but processes a different slice of the minibatch – simple and “embarrassingly parallel”
- ▶ Each step: every machine computes its own local gradient $\mathbf{g}_t^k$, then all machines combine them via an **all-reduce** sum, $\mathbf{g}_t = \sum_{k=1}^K \mathbf{g}_t^k$, before updating their (shared) parameters
- ▶ If every machine waits for this combined gradient before updating, training is **synchronous** (equivalent to training on one machine with a bigger batch, just faster); if machines update immediately using only their own local gradient, it's **asynchronous** (“hogwild” training) – faster per step, but not guaranteed to behave like standard stochastic gradient descent

Regularization: Keeping Big Models from Overfitting

- ▶ Large DNNs can have millions of parameters – controlling overfitting matters. A few complementary strategies are:
- ▶ **Early stopping** – simply stop training once validation error starts to rise, limiting how much information flows from the training data into the parameters
- ▶ **Weight decay** – put a Gaussian prior on the weights, $\mathcal{N}(\mathbf{w} \mid \mathbf{0}, \alpha^2 \mathbf{I})$; and use maximum a posteriori estimation (equiv. to ℓ_2 penalty that shrinks weights to 0)
- ▶ **Sparse DNNs** – ℓ_1 regularization or similar methods drive many weights to exactly zero, enabling model compression
- ▶ **Dropout** – randomly zero out a unit's outgoing connections with probability p during training
- ▶ **Bayesian neural networks** – rather than a single best θ , average predictions over a whole posterior distribution of plausible parameter settings

References

- ▶ [ESLII](#): Chapter 11.3 - 11.5
- ▶ [PML](#): Chapter 13
- ▶ [PRML](#) Chapter 5